Lahore University of Management Sciences
Syed Babar Ali School of Science and Engineering (SBASSE)
AI651 – Deep Learning for Space, Time and Graphs
Fall 2026

Assignment 2 – Report

Name: Hasan Amjad **Roll No.:** 25280042
GitHub repository: https://github.com/HasanAmjad/convlstm-motion-vs-memory

---

Question 1 – River Forecasting Proposal

1. Architecture diagram

All sources are placed on one space–time grid (places × ten-minute steps). A causal temporal convolution summarises what has happened at each place. Then, four rounds of upstream attention pass information down the river, with a learned travel delay. A small head predicts each station's change in level over the next hour.

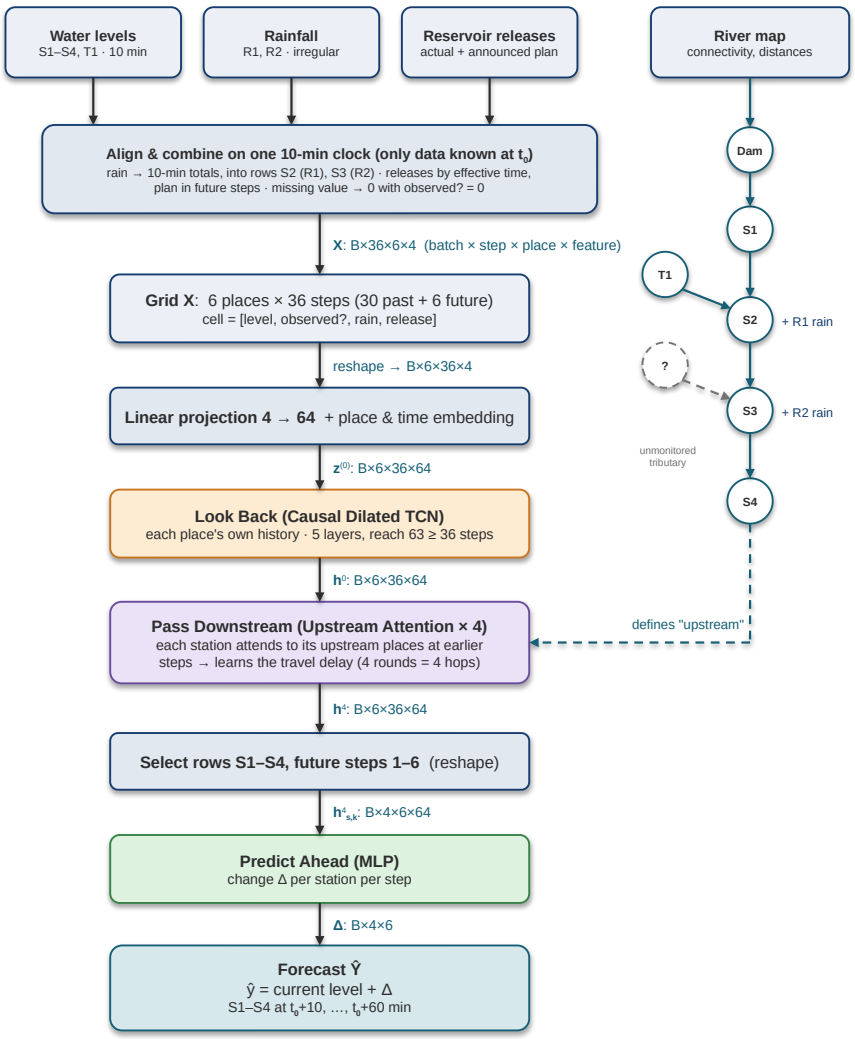


Figure 1: Proposed forecasting system.

2. Assumptions

Before stating assumptions, we list the facts given in the brief.

F1 levels at S1–S4 and T1 every 10 minutes;
F2 rainfall at R1, R2 with timestamps and gauge locations;
F3 recorded releases, and future release schedules with announcement and effective times;
F4 station locations, connectivity and river distances;
F5 a monitored tributary (T1) joins between S1 and S2, an unmonitored one between S2 and S3;
F6 stations are unevenly spaced and travel times change with conditions;
F7 future rainfall, future levels, exact travel times and the unmonitored inflow are not available.

Based on what the brief leaves unspecified, we make the following assumptions. Source markers [1]–[6] refer to the list below the table.

Table 1: Assumptions, why each is reasonable, and its effect on the design.

	Assumption	Why reasonable	Effect on design
Data assumptions (record, timing and quality the brief leaves open)			
A1	Multi-year record with several high-water events.	Deep hourly rainfall–runoff models may need ∼15 years to beat simpler models [4].	Chronological split by year; small model (A16); always compared with simple baselines (Section 4).
A2	One clock (UTC); level readings are instantaneous values.	Standard telemetry practice.	Snap to nearest 10-min mark (±2 min), else missing.
A3	Each rain reading is the total since the previous one.	Tipping-bucket gauges tip at a fixed depth (∼0.25 mm) [2].	Sum into 10-min bins.
A4	Readings are recorded every 10 min (F1) but sent in batches, at least hourly; the newest can be up to ∼1 h old at issuance.	Gauges commonly record every 15 min and transmit about hourly, more often in floods [1].	Issue on time; readings not yet received are missing; training simulates delays up to 1 h (Section 5).
A5	Data contain gaps, spikes and flat-lines, but no long-term drift.	Common sensor faults.	Quality screen; *observed?* flag per cell.
A6	Release schedules are archived with announcement and effective times.	The authority records announcements (F3).	Train on plans *as announced* (no leakage).
A7	Dam-to-S4 travel $\leq\sim$4 h at low flow; rain response mostly within ∼5 h.	Short mountain river (handout figure); checked by the lag of best upstream–downstream correlation.	History window $H = 30$ steps (5 h).
A8	Each station has alert levels θ_s (or high percentiles of the record).	Authorities define action/minor/moderate/major flood stages [3].	Larger loss weight above θ_s.
River-dynamics assumptions (how the model treats the river)			
A9	Water moves only downstream; no backwater effect within an hour.	Steep river with free-flowing junctions.	Attention restricted to upstream places (generic spatial blocks look both ways).

	Assumption	Why reasonable	Effect on design
A10	Travel time is positive and varies with flow: usually shorter as the river rises in its channel, but can lengthen once water spreads onto the floodplain.	Wave speed $\approx \frac{5}{3}\times$ flow velocity in a channel [5], but may fall above bankfull where there is floodplain storage [6].	Delay learned by state-dependent attention, not fixed.
A11	R1 drains to the S1–S2 reach; R2 to the unmonitored tributary (to S3).	Gauge positions in the handout figure.	R1 added to row S2, R2 to row S3.
A12	Unmonitored inflow is driven mainly by rain on its catchment, plus slow baseflow.	Typical of small tributaries.	R2 as proxy; remaining error widens uncertainty at S3/S4.
A13	Releases are the only controlled inflow; withdrawals are negligible.	None mentioned.	No extra input rows.
A14	Without a new announcement, the current release continues.	Operators change releases by announcement.	Future Dam cells hold the current rate.
A15	Gauge datums and channel shape are stable over the record.	No changes mentioned.	One model for the whole record; retrain if changed.
A16	The same learned update rule applies at every station; reaches differ only through place embedding and distance.	Few stations and limited flood data; sharing pools examples.	Shared TCN and attention weights.

Sources. [1] USGS, How often are real-time streamflow data updated?; [2] AMS Glossary, Tipping-bucket rain gauge; [3] NWS, Hydrology definitions; [4] Evaluation of LSTM vs. conceptual models for hourly rainfall–runoff simulations with varied training period lengths (2025); [5] Colorado State Univ., CE717 notes, Unsteady flow in open channels; [6] EGU General Assembly 2017, abstract EGU2017-9406 (wave celerity vs. discharge).

3. Forecasting task and data specification

Issuance and horizon. A forecast is issued every 10 minutes at time t_0 (:00, :10, ...). It predicts the level at S1–S4 at lead times $t_0 + 10k$ min, $k = 1, \ldots, 6$: a 4×6 output per issuance.

History window. The model sees the last $H = 30$ steps (5 h, $t_0 - 290$ min to t_0), which covers the assumed travel and rainfall-response times (A7). The delay within the window is learned.

Grid. All inputs share one grid: $T = H + 6 = 36$ columns ($\tau = -29, \ldots, 0$ past; $\tau = 1, \ldots, 6$ future) and $P = 6$ rows (Dam, S1, T1, S2, S3, S4). Each cell holds $F = 4$ features $[\ell, m, r, q]$: level, observed flag, rain and release. $m = 1$ only if every measured value in the cell has arrived. In the Dam row, past columns hold actual releases and future columns the announced plan; column position separates the two.

Table 2: Inputs and outputs for one batch. B = number of issuances (forecast times) in the batch.

Item	Shape	Axes	Time coverage
Grid $\mathbf{X}$	$B \times 36 \times 6 \times 4$	issuance, column τ, place, feature	$t_0 - 290$ min … $t_0 + 60$ min
level ℓ	(in $\mathbf{X}$)	rows S1–S4, T1	$\tau \le 0$ as observed; $\tau > 0$ empty
observed flag m	(in $\mathbf{X}$)	all rows	1 if every measured value has arrived; 0 otherwise and for $\tau > 0$
rain r	(in $\mathbf{X}$)	R1 in row S2, R2 in row S3	$\tau \le 0$; $\tau > 0$ empty
release q	(in $\mathbf{X}$)	Dam row	$\tau \le 0$ actual; $\tau > 0$ plan announced by t_0
Calendar $\mathbf{C}$	$B \times 36 \times 4$	issuance, column, sin / cos of time of day and day of year	same as $\mathbf{X}$
River map $\mathcal{E}$	5 edges, each with distance d_{ji}	Dam→S1, S1→S2, T1→S2, S2→S3, S3→S4	fixed
Output $\hat{\mathbf{Y}}$	$B \times 4 \times 6$	issuance, station S1–S4, lead time k	$t_0 + 10$ … $t_0 + 60$ min
Target $\mathbf{Y}$, mask $\mathbf{M}$	$B \times 4 \times 6$	as $\hat{\mathbf{Y}}$ (training only)	same

Alignment across sampling times. All sources use one 10-minute clock. A level reading enters column τ if its timestamp lies within ± 2 min of $t_0 + 10\tau$; otherwise the cell is missing. Rain totals (A3) are summed over $(t_0 + 10(\tau - 1),\, t_0 + 10\tau]$ by observation time. Releases are averaged per interval and placed by effective time.

Alignment across locations. Values are not interpolated in space. Each source maps to one place row (rain via A11). River distances enter as edge features in Pass Downstream, so unequal spacing is handled by distances and learned delays, not by treating rows as evenly spaced neighbours.

Information available at issuance. A cell is filled only if its value is known at t_0: a measurement must have arrived, and a plan must have been announced, by t_0. Future levels and rainfall are never available; their cells stay empty with $m = 0$. The only filled future cells are the Dam row's announced releases.

4. Model computations and design justification

Notation. Places $p \in \{\text{Dam}, \text{S1}, \text{T1}, \text{S2}, \text{S3}, \text{S4}\}$; columns $\tau \in \{-29, \ldots, 6\}$; stations $s \in \{\text{S1}, \ldots, \text{S4}\}$; lead times $k = 1, \ldots, 6$; hidden width $d = 64$. $\mathbf{x}_{p,\tau} = [\ell, m, r, q] \in \mathbb{R}^4$ is one grid cell, $\mathcal{U}(i)$ the set of places directly upstream of i (from $\mathcal{E}$), and d_{ji} the river distance from j to i. Levels are standardised per station using training statistics.

Input projection.

$$\mathbf{z}^{(0)}_{p,\tau} = W_{\text{in}}\, \mathbf{x}_{p,\tau} + \mathbf{e}_p + \mathbf{c}_\tau, \qquad \mathbf{z} \in \mathbb{R}^d, \tag{1}$$

where $\mathbf{e}_p$ is a learned place embedding and $\mathbf{c}_\tau$ a time embedding (calendar features plus column position, so future columns are distinguishable).

Look Back (Causal Dilated TCN), applied to each place with shared weights. For layers $l = 1, \ldots, 5$ with kernel size 3 and dilation $\delta_l = 2^{l-1}$:

$$\mathbf{z}^{(l)}_{p,\tau} = \mathbf{z}^{(l-1)}_{p,\tau} + \text{ReLU}\Big(\textstyle\sum_{a=0}^{2} A^{(l)}_a\, \mathbf{z}^{(l-1)}_{p,\,\tau - a\delta_l} + \mathbf{b}^{(l)} \Big), \tag{2}$$

with zero padding before $\tau = -29$. The convolution is causal (columns $\leq \tau$ only). Its receptive field is $1 + 2(1 + 2 + 4 + 8 + 16) = 63 \geq 36$ columns, so each column sees the full window. Set $\mathbf{h}^0_{p,\tau} = \mathbf{z}^{(5)}_{p,\tau}$.

Pass Downstream (Upstream Attention $\times 4$), $R = 4$ rounds with separate weights per round. For round r and each place i with $\mathcal{U}(i) \neq \emptyset$:

$$\mathbf{q}_{i,\tau} = W_Q^{(r)} \mathbf{h}^{r-1}_{i,\tau}, \qquad \mathbf{k}_{j,\tau'} = W_K^{(r)} \mathbf{h}^{r-1}_{j,\tau'} + \mathbf{u}^{(r)}_{\tau-\tau'} + \mathbf{w}^{(r)} d_{ji}, \qquad \mathbf{v}_{j,\tau'} = W_V^{(r)} \mathbf{h}^{r-1}_{j,\tau'}, \tag{3}$$

$$\alpha_{i\tau,\, j\tau'} = \underset{j \in \mathcal{U}(i),\ \tau' \leq \tau}{\operatorname{softmax}} \left(\mathbf{q}_{i,\tau}^\top \mathbf{k}_{j,\tau'} / \sqrt{d} \right), \qquad \mathbf{m}_{i,\tau} = \textstyle\sum_{j,\tau'} \alpha_{i\tau,\, j\tau'}\, \mathbf{v}_{j,\tau'}, \tag{4}$$

$$\tilde{\mathbf{h}}_{i,\tau} = \mathrm{LN}(\mathbf{h}^{r-1}_{i,\tau} + W_O^{(r)} \mathbf{m}_{i,\tau}), \qquad \mathbf{h}^r_{i,\tau} = \mathrm{LN}(\tilde{\mathbf{h}}_{i,\tau} + \mathrm{FFN}^{(r)}(\tilde{\mathbf{h}}_{i,\tau})). \tag{5}$$

$\mathbf{u}_{\tau-\tau'}$ is a learned lag embedding and $\mathbf{w} d_{ji}$ a distance term. Places without upstream neighbours (Dam, T1) keep $\mathbf{h}^r = \mathbf{h}^{r-1}$. Four heads are used; one is written for clarity.

Predict Ahead (MLP). Each station reads its own representation at future column k:

$$\Delta_{s,k} = g(\mathbf{h}^4_{s,k}), \qquad \hat{y}_{s,k} = \tilde{\ell}_{s,0} + \sigma_s\, \Delta_{s,k}, \tag{6}$$

where g is a two-layer MLP shared by all stations (the place embedding separates them), $\tilde{\ell}_{s,0}$ is the latest valid level at s, and σ_s the training standard deviation of one-hour level changes at s.

Training objective. Weighted, masked mean squared error, as in the tutorial:

$$\mathcal{L} = \frac{\sum_{b,s,k} M_{b,s,k}\, w_{b,s,k}\, (\hat{y}_{b,s,k} - y_{b,s,k})^2 / \sigma_s^2}{\sum_{b,s,k} M_{b,s,k}\, w_{b,s,k}}, \qquad w_{b,s,k} = 1 + \lambda \mathbb{1}[y_{b,s,k} \geq \theta_s], \tag{7}$$

with $M = 0$ for missing targets, $\lambda = 4$ (tuned on validation) and alert levels θ_s (A8). Training uses Adam, early stopping on validation error and a chronological split (A1). 10–20% of cells, and occasionally a whole row, are masked at random so the model learns to forecast with missing data. MSE targets the conditional mean, so peaks are smoothed when information is missing; the high-water weight counters this (Table 3). *Optional extension:* g outputs the 10/50/90% quantiles, trained with pinball loss, to give a forecast range.

Why this design.

- *Upstream paths are required.* A station's own history cannot show water already in transit (companion notebook, Part 1). Pass Downstream gives each station a path to every upstream source, with neighbours defined by the river map (F4), not by index order. Four rounds cover the longest path, Dam→S1→S2→S3→S4.
- *Delay must be learned, not fixed.* Travel time changes with flow (F6, A10). Attention weights are recomputed for each input, so the same model can look 50 min back on a calm day, 25 min back as the river rises, and further back if an overbank flood slows the wave. A fixed kernel would impose one delay.
- *TCN for local history.* A causal TCN handles the look-back step; an LSTM/GRU would be an equally reasonable choice. The TCN was chosen for its exact receptive field ($63 \geq 36$ steps), parallel computation, and consistency with the causal temporal blocks in the companion notebook. It also extracts local shape (rising limb, peak), which single-value attention tokens cannot represent.
- *Announced releases on the same grid.* Covariates help only when they cover the forecast horizon (PA1). The plan in the Dam row's future cells reaches S1 and onward through the same blocks.
- *Predicting the change.* Levels persist over one hour. Predicting Δ makes persistence the default, so the network learns only rises and falls.

How each source reaches each forecast.

- *Reservoir releases* (past and planned) enter at the Dam row and reach S1, S2, S3 and S4 in rounds 1–4.
- *Upstream conditions*: each station attends to its upstream neighbours, whose states already carry information from further upstream.
- *Monitored tributary*: T1 is a direct upstream neighbour of S2, and through S2 of S3 and S4.
- *Rainfall*: R1 enters S2's row (local inflow); R2 enters S3's row as the proxy for the unmonitored tributary (A11, A12). The TCN learns the rain-to-river delay.

Physical assumptions vs. learned mappings.

Imposed by design (physical assumptions)	Learned from data
Flow direction and connectivity: information moves only downstream (A9)	Travel time and how it changes with flow (attention over lags)
Causality: no column uses later columns; nothing unknown at t_0	Attenuation and spreading of flood waves between stations
Where each source enters the river (Dam→S1, T1→S2, rain rows; A11)	Rainfall-to-level response and its delay
Positive, bounded delay: at most 5 h (A7)	Unmonitored inflow, inferred from R2 and the S2–S3 difference
Continuity: forecast = current level + change	How reliable announced releases are (plan in future Dam cells, actuals in past cells)

No separate physical simulator is used; physics enters through the structure above. Uncertainty is handled by the optional quantile output and the missing-data treatment in Sections 5 and 6.

Baselines for evaluation. Persistence ($\hat{y}_{s,k} = \tilde{\ell}_{s,0}$) and an Autoformer with all places mixed into one token per step (as in PA1). Ablations: no Pass Downstream (each station alone), no future plan, and fixed instead of attended delays.

5. Operation and exceptional conditions

(i) Missing observations. A missing level or rain value is set to 0 with $m = 0$, which separates "missing" from "zero". Random masking in training (Section 4) teaches the model to rely on upstream and neighbouring rows. If a station's current level is missing, the forecast is anchored on its latest valid value $\tilde{\ell}_{s,0}$ and marked lower-confidence. Missing targets are excluded from the loss ($M = 0$). If a station has been missing for more than two hours, forecasts are still issued, flagged to operators, with a wider range.

(ii) Delayed observations. The forecast is issued on time at t_0 with whatever has arrived; values not yet received are missing. A reading is placed by its *observation* timestamp but used only if its *arrival* time is $\leq t_0$. A late reading fills its cell at the next issuance; since one forecast takes milliseconds, the current forecast can also be re-issued immediately. Training simulates delays of up to 1 h (A4), matching the gaps seen in operation.

(iii) Announced future releases. The latest schedule announced by t_0 fills the Dam row's future columns ($\tau = 1, \ldots, 6$) by effective time. A schedule announced after t_0 is not used, even if it takes effect earlier. Without an announcement, the current rate continues (A14). Training uses plans as announced at each past t_0 (A6).

(iv) Scheduled vs. actual releases. Once a release is recorded, its past cell holds the actual value, which overrides the plan. Training pairs announced plans with the levels that followed, so the model learns how far plans can be trusted. If the actual release departs from the plan

beyond a set threshold over the last 30 min, operators are alerted and the range widens. Because a forecast is cheap, operators can also run "what-if" forecasts for alternative plans.

Feasibility every 10 minutes. One forecast processes $6 \times 36 = 216$ cells. The TCN costs about $216 \times 5 \times 3 \times 64^2 \approx 1.3 \times 10^7$ multiply–adds; the four attention rounds about $4 \times 36 \times 72 \times 64 \times 2 \times 4 \approx 5 \times 10^6$; the model has about 2×10^5 parameters. This runs in well under 10 ms on a standard CPU, and alignment needs a few database queries, so a ten-minute cycle leaves a large margin. Training runs offline and is repeated periodically (e.g. monthly or after a major flood).

6. Uncertainty, limitations, and additional information

Table 3: Uncertainties and limitations, their effect on forecasts, and our response.

Issue	Effect on forecasts	Response / unresolved
Unmonitored tributary	S3 and S4 under-forecast in storms that R2 misses.	R2 as proxy (A12); S2–S3 difference; wider range at S3/S4. *Unresolved:* storms missed by R2.
Changing travel times	Peak timing error: early (rising river) or late (overbank flow, A10); worst in rare large floods.	Flow-dependent lag attention; high-water weighting. *Unresolved:* floods outside the record.
Out-of-record floods	Peaks under-predicted; neural networks extrapolate poorly.	Flag out-of-range inputs; fall back to persistence plus upstream rise.
Future and unobserved rain	Rain after t_0, or missed by R1/R2, raises S2/S3 unexpectedly.	Inherent at a one-hour horizon; wider range; rain nowcasts requested.
Data gaps and batching	Up to 1 h missing before each batch (A4); spikes or stale values.	Masks; training with simulated delays and masking; quality screen.
Release deviations	S1, then S2–S4, wrong when operations depart from the plan.	Recorded actuals override the plan; deviation alert (Section 5).
Peak smoothing	MSE predicts the conditional mean, which flattens peaks under uncertainty.	High-water weighting; optional quantile output.
Short or ageing record	Few floods, so a deep model may not beat simple methods [4]; channel or rule changes age the data.	Small shared-weight model; deploy only if it beats persistence on validation floods; monitor and retrain.

Verification of learned delays. A path does not guarantee that the model uses it; in the companion notebook only one attention head learned to read the source. Attention maps are therefore inspected after training: S3 should attend to S2 at a lag near the travel time, and the lag should shorten as the river rises. If not, the path exists but is unused, and the model is retrained or the design revised.

Table 4: Additional information requested from the authority.

Request	How it helps	Usable without it?
Gauge on the unmonitored tributary (temporary is enough)	Removes the main blind spot; calibrates the R2 proxy.	Yes; wider range at S3/S4.
Archived release schedules with announcement times	Training on plans as known at t_0 (A6).	Partly; actuals as a stand-in overstate plan reliability.
Flood alert levels and flood reports	Loss weights; event-based evaluation.	Yes; percentiles instead.
Rain radar or nowcasts	Rain expected in the next hour.	Yes; current design.
Rating curves or discharge	Water-balance checks; estimate of unmonitored inflow.	Yes.
Real-time telemetry and delay logs	Removes pre-batch gaps; realistic delay simulation (A4).	Yes; less sharp before each batch.
Channel and floodplain geometry	Identifies overbank flow and longer travel times (A10).	Yes; delay still learned.
Catchment boundaries for R1, R2	Confirms the rain-to-row mapping (A11).	Yes; A11 as stated.
Reservoir level and spillway rules	Anticipates unscheduled spills.	Yes; spills seen only at S1.

Question 2 – Motion or Memory?

Hardware: MacBook Pro (2021), Apple M1 Pro, 32 GB (PyTorch 2.15.0, `mps` backend). Code: `Question2/` in the repository.

1. Setup

Table 1. Transform and window checks.

	Check	Expected	Yours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0 (`torch.equal`: True)
(b)	$\max \lvert R^4(x) - x \rvert$	0	0 (`torch.equal`: True)
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0 (`torch.equal`: True)
(d)	bright pixel of $M(p)$	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$	(7, 43, 10)	(7, 43, 10)
(f)	1× input / target	1–5 / 6–10	1–5 / 6–10
(g)	2× input / target	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h)	3× input / target	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

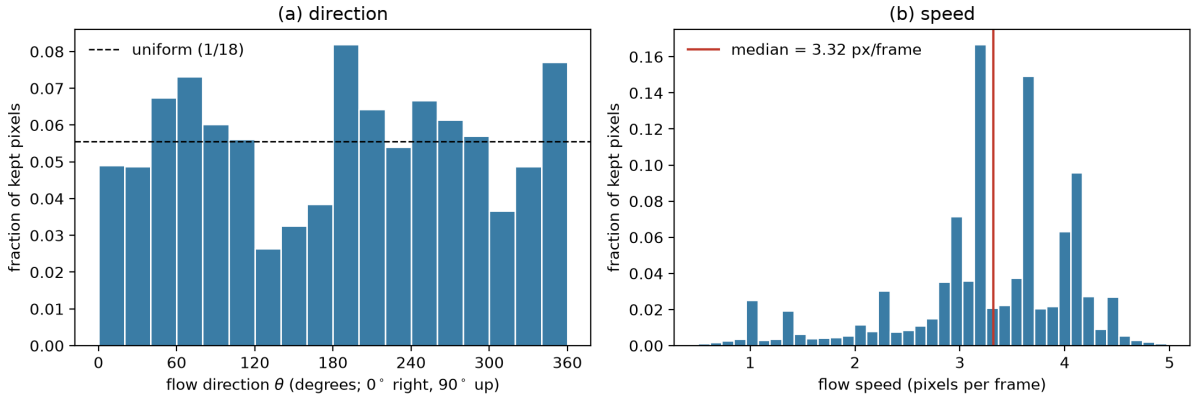


Figure 1. Farneback flow on the first 50 training videos (950 frame pairs, 103,050 pixels kept). (a) Direction, as a fraction of kept pixels; dashed line = uniform (1/18). (b) Speed; median 3.32 px/frame.

Q2.1. Mirroring maps a direction θ to $180° - \theta$ and a 90° rotation maps it to $\theta + 90°$. Figure 1(a) has mass in every 20° bin (0.026–0.082 against a uniform 0.056), and the unevenness has no clear axis, so it looks like sampling noise from 50 videos rather than a preferred direction. A mirrored or rotated histogram would look much the same, so mirrored and rotated *motion* is already familiar. A typical speed is 3.3 px/frame (10–90%: 2.1–4.1). At 2× and 3× the digits move about 6.6 and 10.0 px/frame, above almost every training speed, so faster motion is new. Flow is unreliable where digits overlap, but overlaps are a small share of the kept pixels and would add scattered errors in direction and speed rather than shift the whole histogram, so the answer would not change.

Q2.2. Mirror and rotation keep the motion statistics (same speeds, directions already covered) but change the digits' appearance, rotation more than mirror. Speed-up keeps the appearance but changes the motion statistics. Each test therefore changes one factor, so a failure under mirror or rotation points to appearance, and a failure under speed-up points to motion.

2. Baseline: ConvLSTM

Q2.3. One cell of the folded 16×16 grid is 4 original pixels. A 3×3 convolution moves information by at most one cell (4 px); the 1×1 output convolution and pixel (un)shuffle move nothing, and the cell-state update is element-wise. Within one step τ, the longest path is u_τ (or $h^{(1)}_{\tau-1}$) $\to$ layer-1 convolution $\to$ $h^{(1)}_\tau$ $\to$ layer-2 convolution $\to$ $h^{(2)}_\tau$ $\to$ $\hat{x}_{\tau+1}$: two 3×3 convolutions, so 2 cells = **8 px per time step**. The path from $h^{(2)}_{\tau-1}$ crosses only the layer-2 convolution (4 px).

Table 2. Test error at $1\times$ ($t = 1$), mean (seed 0, seed 1).

	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Params
A	0.0172 (0.0176, 0.0167)	0.0205 (0.0210, 0.0200)	0.0228 (0.0233, 0.0222)	0.0244 (0.0250, 0.0238)	0.0260 (0.0266, 0.0255)	0.0222 (0.0227, 0.0216)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

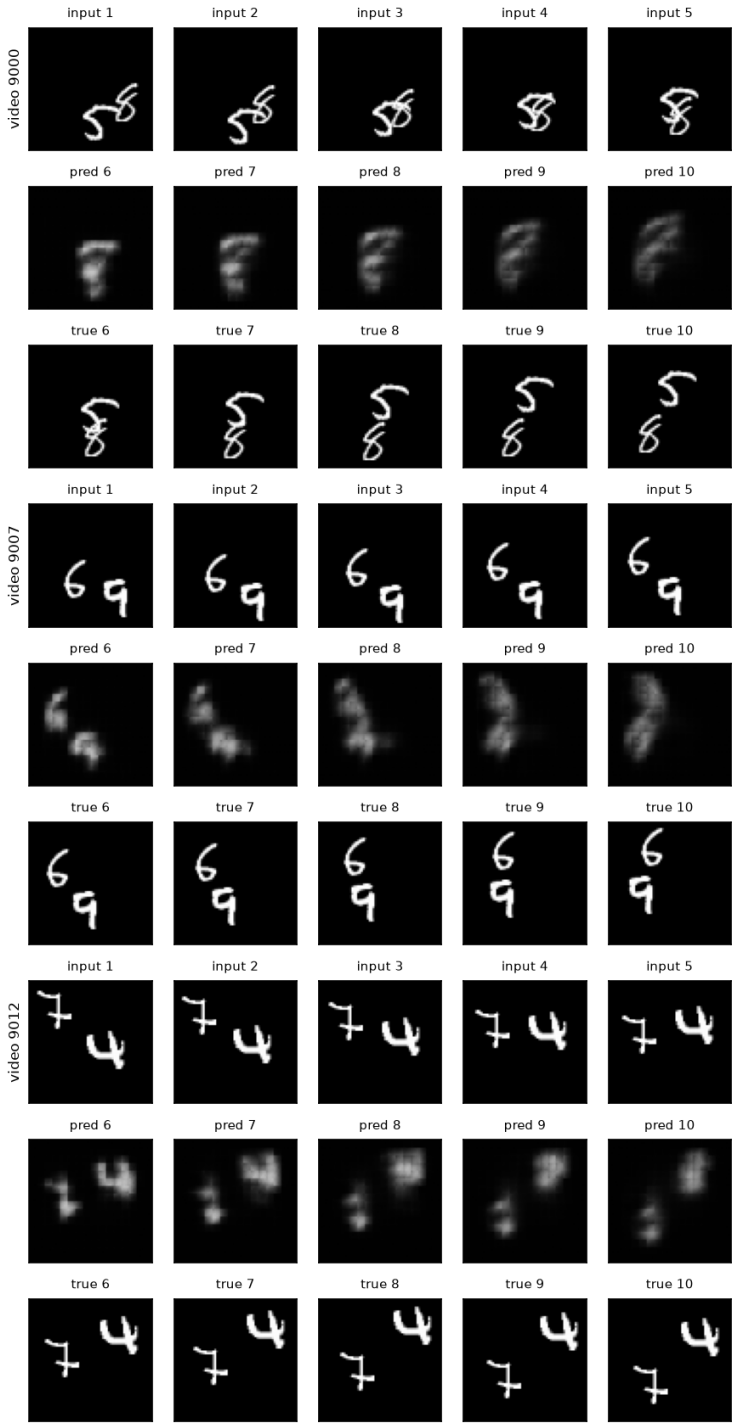


Figure 2. Model A (seed 0) on three test videos: input frames 1–5, prediction and ground truth for frames 6–10. In video 9000 a 5 and an 8 overlap in the input and separate in the target.

Q2.4. Model A beats both references at every horizon: its $E^{(5)} = 0.0222$ is about half of Blank (0.0426) and a third of Copy-last (0.0641), so it does more than reproduce the last frame. Its error grows steadily, from 0.0172 at $h = 1$ to 0.0260 at $h = 5$ (about 1.5×), because each prediction is fed back as the next input and errors accumulate. Blank stays flat and Copy-last grows faster (0.0492 to 0.0732). Figure 2 shows why the error stays well above zero: positions are tracked roughly, but the digits blur within two or three steps and lose their identity. When

digits overlap (video 9000), the model predicts one merged blob that drifts as a whole; it never separates the 5 and the 8, which in the true future move apart in opposite directions.

Q2.5 (written before testing). Reach of model A (Q2.3): about 8 px per time step. Typical training speed (Figure 1): 3.3 px/frame (10–90%: 2.1–4.1). **Mirror:** passes, mostly. Mirrored motion is already common in training (Figure 1a covers all directions), so the motion should carry over. Mirrored digits look unfamiliar, so expect some blurring of digit shape, and C_M small but not zero. **Rotation:** passes the motion part but worse than mirror. Rotated motion is also familiar, but a sideways digit is less like any training digit than a mirrored one, so $C_R > C_M$. **Speed:** 2× (≈ 6.6 px/frame) is inside the 8 px reach but above every training speed, so predictions degrade but still follow the digits. 3× (≈ 10 px/frame) is beyond the reach, so the model breaks down there: digits stall or fade near their last input position.

3. Testing the Baseline

Table 3. Consistency gaps and transformed errors at 1×, mean (seed 0, seed 1).

	$E^{(5)}$	C_M	E_M	C_R	E_R
A	0.0222 (0.0227, 0.0216)	0.00176 (0.00158, 0.00195)	0.0228 (0.0233, 0.0223)	0.00289 (0.00260, 0.00318)	0.0239 (0.0242, 0.0236)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

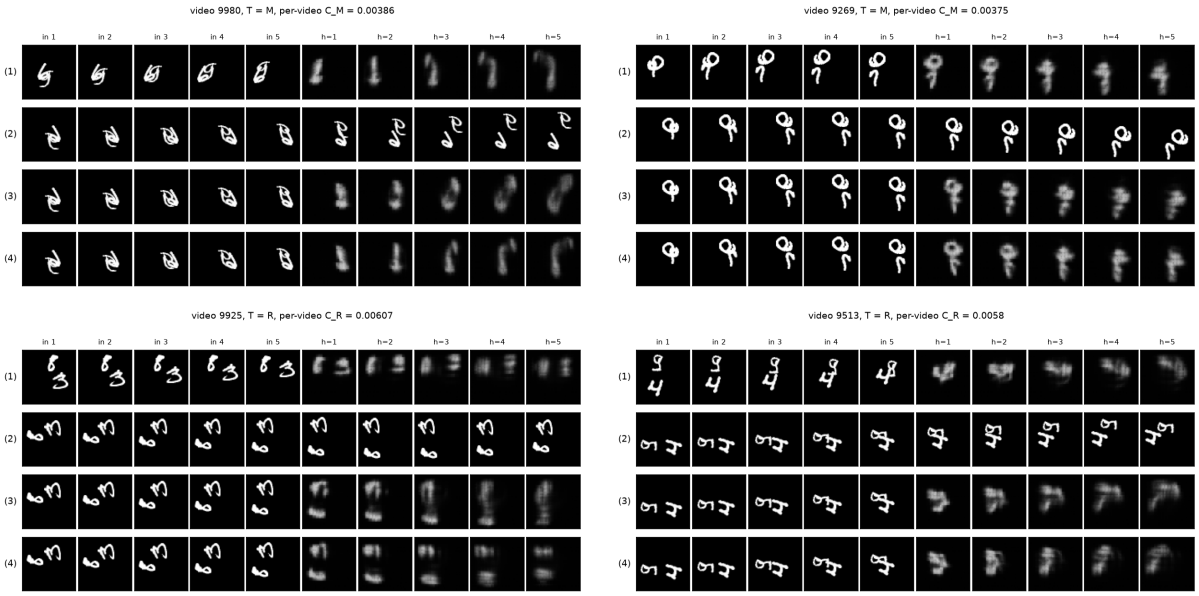


Figure 3. Test videos with the largest per-video C_M (top) and C_R (bottom), seed-0 model A. Columns 1–5: input; 6–10: horizons 1–5. Rows: (1) $X \,|\, f(X)$, (2) $T(X) \,|\, T(Y)$, (3) $T(X) \,|\, f(T(X))$, (4) $T(X) \,|\, T(f(X))$. GIFs: `Question2/gifs/fig3_A_M_video9980.gif`, `fig3_A_M_video9269.gif`, `fig3_A_R_video9925.gif`, `fig3_A_R_video9513.gif`.

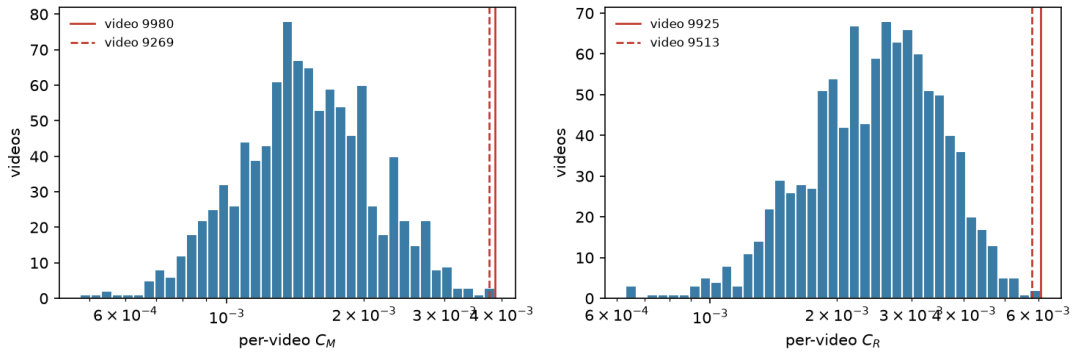


Figure 4. Per-video C_M and C_R of seed-0 model A over the 1,000 test videos (log scale). Red lines: the videos in Figure 3.

Q2.6. Blank ignores the input and Copy-last only repeats it, so both commute with any rearrangement of pixels and have $C_M = C_R = 0$, yet their errors (0.0426 and 0.0641) are the worst in Table 3. A small C_T only says that the model treats the transformed clip the same way as the original, not that it treats either one correctly: a model can be consistently wrong. E_T measures correctness on the transformed clip, so the two must be read together.

Q2.7. Video 9925 under rotation (Figure 3, bottom left). In the rotated clip (row 2) one digit sits at the top and the other at the bottom left, and both drift slowly. Predicting from the rotated input (row 3), the model stretches the top digit downwards into a vertical streak that by $h = 3$ reaches the lower digit, so the two digits merge into one column and the top digit becomes unrecognisable. Rotating the original prediction (row 4) instead gives two separate horizontal blobs that stay near their last positions and fade. So on the rotated clip the model invents a downward motion for the top digit that is not in the data and loses it in a smear, while on the upright clip it keeps both digits apart. The motion rule it applies depends on what the digits look like.

Q2.8. Yes. For model A, $C_R = 0.00289$ is about 1.6 times $C_M = 0.00176$, and the cost of transforming the clip is larger under rotation ($E_R - E^{(5)} = 0.0017$) than under mirroring ($E_M - E^{(5)} = 0.0006$). By Q2.1 both transformations give motion that is already common in training, so the motion does not explain the gap. By Q2.2 what differs is appearance: a mirrored digit keeps its upright strokes, while a rotated digit lies on its side and is further from anything in training. The difference therefore comes from appearance.

Table 4. Model A at higher speed ($t = 1$), mean (seed 0, seed 1). Blank: $E^{(2)} = 0.0426$, 0.0425, 0.0426 at 1×, 2×, 3×.

	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
A	0.0189 (0.0193, 0.0184)	0.0444 (0.0431, 0.0457)	0.0522 (0.0511, 0.0533)	0.0222 (0.0227, 0.0216)	0.0488 (0.0467, 0.0509)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

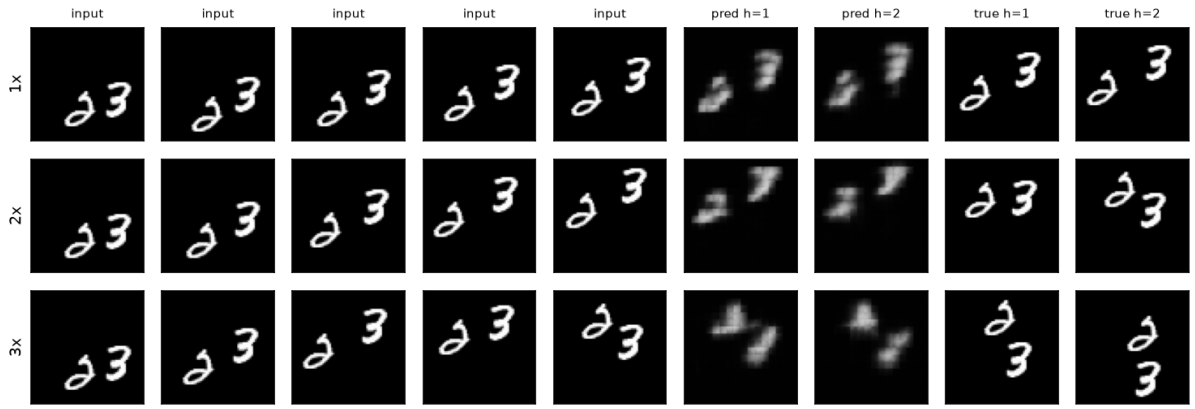


Figure 5. Test video 9026 at 1×, 2×, 3× (seed-0 model A): five input frames, first two predicted and true frames. The digits do not overlap in any displayed true frame.

Q2.9. Model A already falls apart at 2×: $E^{(2)}$ rises from 0.0189 to 0.0444, worse than Blank (0.0425), and reaches 0.0522 at 3×. This does not agree with the reach estimate, which put 2× (6.6 px/frame) inside the 8 px reach. The estimate missed two things. First, 8 px is the most information can move along the single longest path; to follow a digit the model must also detect its displacement, and in layer 1 the new frame and the previous state meet in one 3×3 convolution that spans only one cell (4 px). A 6.6 px jump is not visible there, so the motion has to be both detected and carried forward within the same two-layer budget. Second, the model only learned displacements of about 2–5 px, so even moves within reach are ones it never had to make. In Figure 5, at 1× both digits move correctly but blur. At 2× and 3× the digits stall near their last input position and fade into blobs instead of jumping to the true position. Stalling and fading suggest the model falls back to a blurred copy of the last frame: it does not connect the old and new positions, so it hedges by spreading intensity near where the digit was.

Q2.10. No. Both explanations predict the same thing here: at 2× and 3× the digits move faster than anything model A saw in training (Figure 1b: 2.1–4.1 px/frame for 80% of pixels), and at 3× also faster than its 8 px reach. Since model A was trained on 1× only, the two causes are confounded. To separate them, train the same architecture on all three speeds (model C). If the failure is unfamiliar motion, C should recover at 2× and 3×. If it is reach, C should improve at 2× (within reach) but still fail at 3×, and only a model with larger reach trained on the same speeds (model D) should fix 3×.

4. What Can Data Fix?

Q2.11 (written before training B–D). Model B (symmetry augmentation) will close most of the mirror and rotation gap (lower C_M, C_R and E_M, E_R close to $E^{(5)}$ at 1×) but leave the 2× and 3× failure unchanged. Model C (speed augmentation) will fix 2× and improve 3× without fully fixing it, because 10 px/frame is still beyond its 8 px reach. Model D (larger reach, speed augmentation) will fix 3× as well, and neither C nor D will fix mirror or rotation.

Table 5. Models A–D on the test split, mean (seed 0, seed 1). Hardware: MacBook Pro (2021), Apple M1 Pro, 32 GB, PyTorch `mps`. Training time is per epoch; inference is all 1,000 test videos (1×, $t = 1$, batch 32).

	A	B	C	D
$E^{(5)}$ 1×	0.0222 (0.0227, 0.0216)	0.0228 (0.0235, 0.0221)	0.0279 (0.0276, 0.0283)	0.0275 (0.0279, 0.0272)
C_M	0.00176 (0.00158, 0.00195)	0.00121 (0.00125, 0.00117)	7.94e-4 (7.99e-4, 7.89e-4)	8.35e-4 (9.69e-4, 7.00e-4)
E_M	0.0228 (0.0233, 0.0223)	0.0228 (0.0235, 0.0221)	0.0282 (0.0278, 0.0285)	0.0277 (0.0280, 0.0274)
C_R	0.00289 (0.00260, 0.00318)	0.00141 (0.00156, 0.00126)	0.00198 (0.00173, 0.00223)	0.00192 (0.00171, 0.00213)
E_R	0.0239 (0.0242, 0.0236)	0.0227 (0.0233, 0.0220)	0.0291 (0.0287, 0.0295)	0.0286 (0.0287, 0.0284)
$E^{(2)}$ 1×	0.0189 (0.0193, 0.0184)	0.0191 (0.0198, 0.0183)	0.0241 (0.0239, 0.0243)	0.0247 (0.0248, 0.0246)
$E^{(2)}$ 2×	0.0444 (0.0431, 0.0457)	0.0441 (0.0439, 0.0444)	0.0295 (0.0295, 0.0295)	0.0292 (0.0296, 0.0289)
$E^{(2)}$ 3×	0.0522 (0.0511, 0.0533)	0.0513 (0.0515, 0.0511)	0.0339 (0.0338, 0.0340)	0.0327 (0.0328, 0.0326)
Parameters	480,784	480,784	480,784	480,784
Train (s/epoch)	25.6 (24.8, 26.4)	24.0 (24.2, 23.9)	23.7 (23.6, 23.8)	27.9 (28.0, 27.7)
Inference (s)	0.721 (0.728, 0.713)	0.714 (0.711, 0.716)	0.711 (0.710, 0.711)	0.920 (0.924, 0.917)

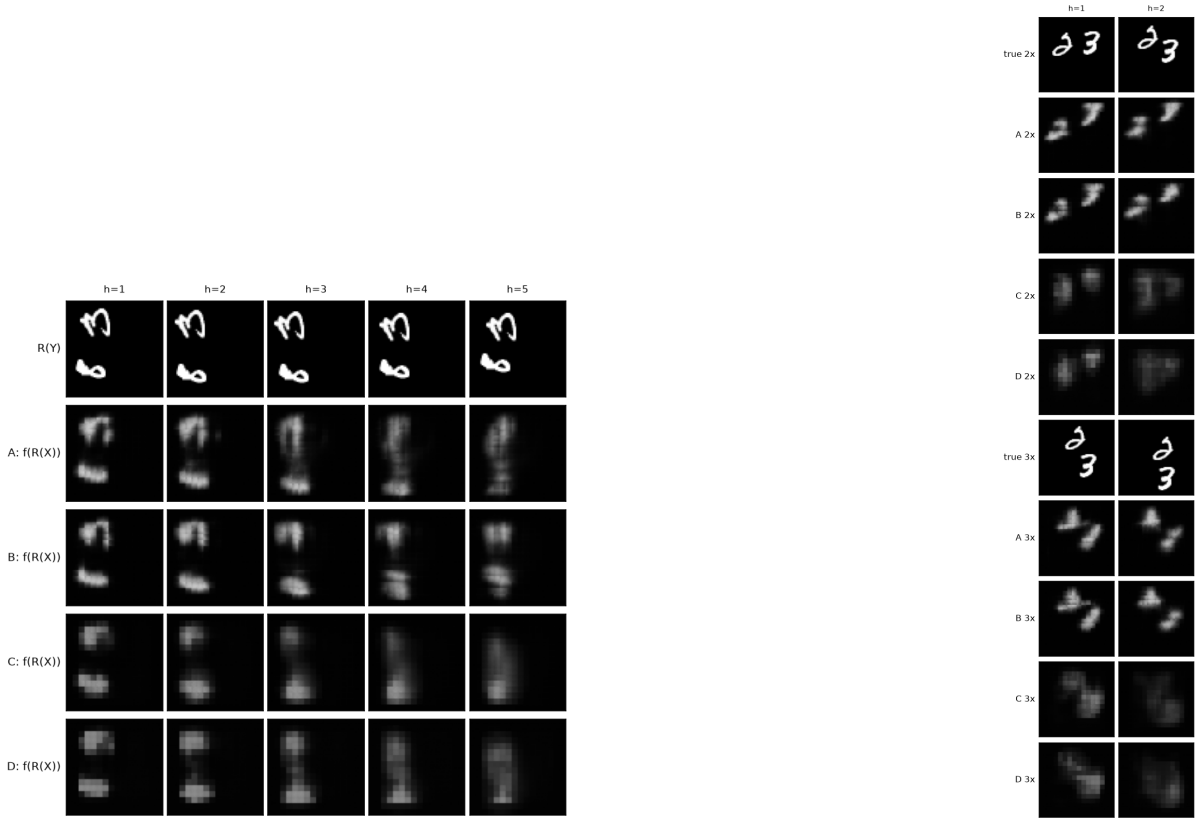


Figure S1. The same clips under models A–D (seed 0). Left: video 9925 rotated, truth $R(Y)$ and $f(R(X))$ for each model. Right: video 9026, first two horizons at 2× and 3×.

Q2.12. Model D is model A with a dilation of 2 in the layer-2 convolution; everything else (patch 4, 64 channels, 3×3 kernels, rollout, training) is unchanged, and dilation adds no weights, so it has 480,784 parameters. Using the conventions of Q2.3: layer 1 moves information by at most 1 cell and layer 2, now spanning ±2 cells, by at most 2. The longest path in one step, u_τ or $h^{(1)}_{\tau-1} \to$ layer 1 $\to$ layer 2 $\to \hat{x}_{\tau+1}$, therefore covers $1 + 2 = 3$ cells = **12 px**, strictly larger than model A's 8 px. Offsets $\{-1, 0, 1\} + \{-2, 0, 2\}$ cover every cell in $-3 \ldots 3$, so no shift within the reach is missing (dilation 2 in both layers would reach only even offsets). Three candidates were trained with seed 0 under model D's protocol and compared on the validation split (`results/d_selection.json`): dilations (1, 2) at 12 px, (1, 3) at 16 px, and 5×5 kernels with 46 channels at 16 px (709,520 parameters). Mean validation $E^{(2)}$ over the three speeds

was 0.0287, 0.0307 and 0.0395, and the 5×5 model never left the blank solution, so (1, 2) was chosen and its seed-0 run is model D's seed 0.

Q2.13. *Rotation (and mirror) gap: appearance.* B changes only the training orientations, halves C_R (0.00289 to 0.00141) and lowers C_M by about a third (0.00176 to 0.00121); E_R falls from 0.0239 to 0.0227, level with its $E^{(5)}$ of 0.0228, while its speed errors are unchanged (0.0441, 0.0513). *Failure at 2×: unfamiliar motion.* C changes only the training speeds and cuts $E^{(2)}$ at 2× from 0.0444 to 0.0295, without any extra reach. *Failure at 3×: mostly unfamiliar motion, with a small reach part.* C cuts it from 0.0522 to 0.0339; D, which adds only reach, lowers it further to 0.0327, about 3.5% (lower in both seeds: 0.0328 against 0.0338, 0.0326 against 0.0340). C did not "still do badly" at 3×: although 10 px exceeds its 8 px one-step reach, the recurrent state can carry a velocity estimate over two steps (16 px), which is why speed training alone helps. Reach still matters as a hard limit: within one update a pixel can only affect outputs within the reach, and data can change the weights but cannot create a connection that the computation does not have. Figure S1 adds a caution: at 2× and 3× C and D lower the error partly by spreading intensity over the likely positions rather than tracking the digits sharply.

Q2.14. Yes. Speed augmentation costs accuracy at 1×: $E^{(5)}$ rises from 0.0222 (A) to 0.0279 (C) and 0.0275 (D), about 25%, and the predictions are blurrier (Figure S1). The same 480,784 parameters and 30 epochs now cover three speed regimes. Symmetry augmentation costs almost nothing (B: 0.0228). The trade is worth it if the forecaster must face unknown speeds: for D, the 3× error falls by 37% and the 2× error by 34% relative to A. If only 1× clips matter, model A remains the most accurate.

5. The Final Model

Exact consistency. For any $h \in G$, substitute $g' = gh$, which runs over all of G as g does, so $g = g'h^{-1}$ and $g^{-1} = h\,g'^{-1}$:

$$\bar{f}(hX) = \frac{1}{8}\sum_{g\in G} g^{-1} f(ghX) = \frac{1}{8}\sum_{g'\in G} h\,g'^{-1} f(g'X) = h\,\bar{f}(X),$$

since h is a linear rearrangement of pixels and commutes with the average. With $h = M$ and $h = R$, $\bar{f}(T(X)) = T(\bar{f}(X))$, so $C_M = C_R = 0$ for any f. Table 6 confirms this to about 10^{-16}.

Choice of final model. Symmetrised D. Table 5 suggested D+B might help: B halved C_R at no cost on 1× clips, and a base model whose eight views agree more could give a sharper average. D+B was therefore trained (both seeds) and compared with D, both symmetrised, on the validation split only, using the same criterion as for model D (mean $E^{(2)}$ over 1×, 2×, 3×, averaged over seeds; `results/final_selection.json`). Symmetrised D scored 0.0286 and symmetrised D+B 0.0288, with D equal or slightly better at every speed, so D was kept. Once the forecaster is symmetrised, symmetry augmentation adds nothing measurable.

Table 6. Symmetrised models, mean (seed 0, seed 1). Training time is that of the underlying model; inference includes all eight transformations, forward passes, inverses and the averaging.

	Symmetrised B	Symmetrised D (final)
$E^{(5)}$ 1×	0.0221 (0.0228, 0.0215)	0.0275 (0.0278, 0.0272)
C_M	8.11e-17 (8.75e-17, 7.46e-17)	5.67e-17 (5.08e-17, 6.25e-17)
E_M	0.0221 (0.0228, 0.0215)	0.0275 (0.0278, 0.0272)
C_R	6.57e-17 (7.14e-17, 6.00e-17)	4.40e-17 (3.96e-17, 4.84e-17)
E_R	0.0221 (0.0228, 0.0215)	0.0275 (0.0278, 0.0272)
$E^{(2)}$ 1×	0.0186 (0.0194, 0.0179)	0.0247 (0.0248, 0.0246)
$E^{(2)}$ 2×	0.0434 (0.0432, 0.0436)	0.0293 (0.0296, 0.0289)
$E^{(2)}$ 3×	0.0503 (0.0505, 0.0501)	0.0327 (0.0327, 0.0326)
Parameters	480,784	480,784
Train (s/epoch)	24.0 (24.2, 23.9)	27.9 (28.0, 27.7)
Inference (s)	5.03 (5.03, 5.03)	6.51 (6.51, 6.50)

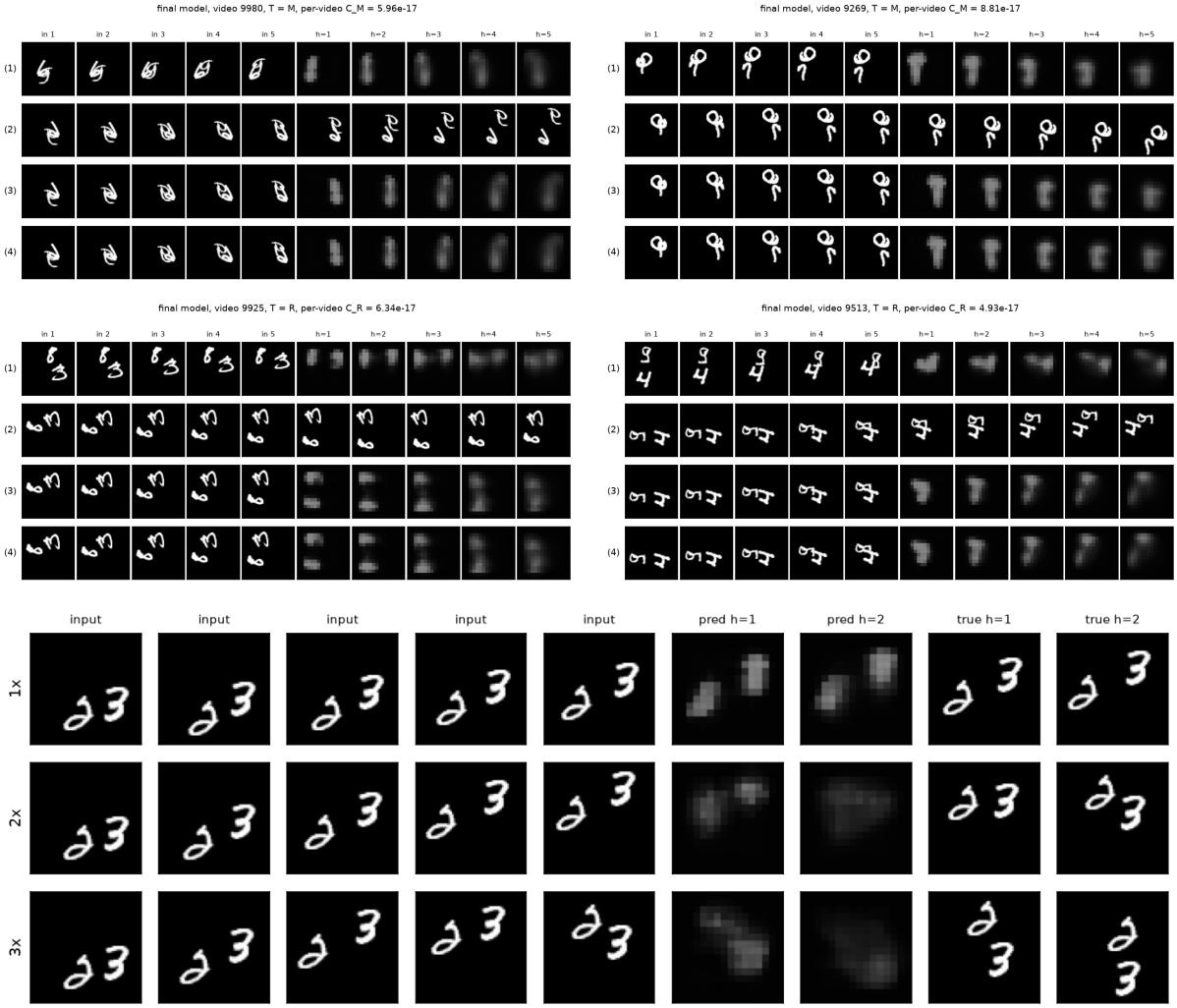


Figure 6. Final model (symmetrised D, seed 0). (a) Top four panels: the videos and rows of Figure 3. GIFs: `Question2/gifs/fig6a_final_M_video9980.gif`, `fig6a_final_M_video9269.gif`, `fig6a_final_R_video9925.gif`, `fig6a_final_R_video9513.gif`. (b) Bottom: video 9026 at 1×, 2×, 3× as in Figure 5.

Q2.15. Learned symmetry (B) lowers the gaps ($C_M = 0.00121$, $C_R = 0.00141$) but does not remove them; the loss only encourages consistency on the training clips. Built-in symmetry is exact ($C_M, C_R \approx 10^{-16}$) because, by the argument above, it holds for any weights. Accuracy on the original clips is not hurt: symmetrised B has $E^{(5)} = 0.0221$ against 0.0228 for B, and symmetrised D matches D (0.0275); on transformed clips $E_M = E_R = E^{(5)}$ by construction. The cost is inference: eight forward passes make it about seven times slower (0.714 to 5.03 s for B, 0.920 to 6.51 s for D). Averaging blurs when the eight views disagree, so built-in symmetry should hurt most when the base model is far from consistent or when the task itself is not symmetric. Here the views already agreed closely, and since squared error is convex, the average is never worse than the average error of the eight views; so it did not hurt on the numbers, although Figure 6 is visibly softer than Figure 3. Making the views agree more before averaging did not sharpen the result either: symmetrised D+B was no better than symmetrised D on validation.

Q2.16. Partly. For video 9925, rows 3 and 4 of Figure 6(a) are now identical ($C_R = 6.34 \times 10^{-17}$): symmetrisation removed the downward streak that model A invented only for the rotated clip, so the motion rule no longer depends on orientation. The prediction is still not right: both digits are coarse blobs that stay near their last positions and by $h = 4$–5 smear into one column, while in the truth they stay apart. Symmetrisation fixed the consistency; speed training and the larger reach do not help this slow clip and add blur.

Q2.17 Final verdict. Yes, the model did partly learn motion, but it cannot be said that it has fully learned how things move. It applies the same rule in all eight orientations exactly, and with speed training its 2× and 3× errors are about a third lower than model A's, so its predictions depend on motion and not only on how the training clips look. However, lower error is not proof of tracking. Convincing evidence would be predicted digit centres following the true trajectories, including bounces, at every speed and orientation, and sensible predictions at untrained speeds like 1.5× or 4×. That evidence is missing: predictions blur within two or three steps, digits lose their identity, and on fast clips (Figure 6b) part of the gain comes from spreading intensity over likely positions. One test it still fails is keeping two overlapping digits apart after they separate (video 9000).

Appendix – Use of Generative AI

Claude (Anthropic) was used to understand different aspects of the assignment, to critically analyse my options, and to compile the report in LaTeX. Every decision was my own, and all code was run and tested locally on my machine.